Step 1: Library import

Python libraries were imported to support data handling, visualisation and machine learning workflow.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import collections
import seaborn as sns
import matplotlib
import sklearn
from sklearn.model_selection import train_test_split
import imblearn
import xgboost
import shap

!pip install imbalanced-learn
!pip install xgboost
!pip install shap
```

```
Requirement already satisfied: imbalanced-learn in c:\users\kamalfd\anaconda3\lib\site-packages (0.13.0)
Requirement already satisfied: numpy<3,>=1.24.3 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn) (2.3.4)
Requirement already satisfied: scipy<2,>=1.10.1 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn) (1.16.
3)
Requirement already satisfied: scikit-learn<2,>=1.3.2 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn)
(1.6.1)
Requirement already satisfied: sklearn-compat<1,>=0.1 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn)
(0.1.3)
Requirement already satisfied: joblib<2,>=1.1.1 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn) (1.4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\kamalfd\anaconda3\lib\site-packages (from imbalanced-learn)
(3.5.0)
Requirement already satisfied: xgboost in c:\users\kamalfd\anaconda3\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\kamalfd\anaconda3\lib\site-packages (from xgboost) (2.3.4)
Requirement already satisfied: scipy in c:\users\kamalfd\anaconda3\lib\site-packages (from xgboost) (1.16.3)
Requirement already satisfied: shap in c:\users\kamalfd\anaconda3\lib\site-packages (0.52.0)
Requirement already satisfied: numpy>=2 in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (2.3.4)
Requirement already satisfied: scipy in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (2.2.3)
Requirement already satisfied: tqdm>=4.27.0 in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (24.2)
Requirement already satisfied: slicer==0.0.8 in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (0.0.8)
Requirement already satisfied: numba in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (0.62.1)
Requirement already satisfied: llvmlite in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (0.45.1)
Requirement already satisfied: cloudpickle in c:\users\kamalfd\anaconda3\lib\site-packages (from shap) (3.0.0)
Requirement already satisfied: colorama in c:\users\kamalfd\anaconda3\lib\site-packages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\kamalfd\anaconda3\lib\site-packages (from pandas->shap) (2.9.
0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\kamalfd\anaconda3\lib\site-packages (from pandas->shap) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\kamalfd\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\kamalfd\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas->s
hap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\kamalfd\anaconda3\lib\site-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\kamalfd\anaconda3\lib\site-packages (from scikit-learn->shap)
(3.5.0)
```

Step 2: Dataset Loading & Inspection

The Portuguese Bank Marketing dataset is a structured dataset consisting of 41,188 observations and 21 variables. An initial data quality inspection was carried out to identify dataset dimensions, column names, data types and target classes.

In [2]:
```
# STEP 2: DATA ACQUISITION

# File path to the dataset
FILE_PATH = r"C:\Users\kamalFD\OneDrive\Desktop\data-folder\Bank Marketing Dataset\bank-additional-full.csv"

# Load CSV file into a pandas DataFrame,
# sep=";" this file uses semicolons instead of commas
df = pd.read_csv(FILE_PATH, sep=";")

# Shape of the dataset
print("Shape:", df.shape)

# Inspect data types
print("\nData types:")
print(df.dtypes)

# Check target variable 'y' exists
print("\nIs 'y' in the columns?:", "y" in df.columns)

# Confirm target classes and their counts
print("\nTarget class counts:")
print(df["y"].value_counts())

# Preview the first 20 rows
df.head(20)
```

```
Shape: (41188, 21)

Data types:
age                 int64
job                object
marital            object
education          object
default            object
housing            object
loan               object
contact            object
month              object
day_of_week        object
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome           object
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                  object
dtype: object

Is 'y' in the columns?: True

Target class counts:
y
no     36548
yes     4640
Name: count, dtype: int64
```

Out[2]:

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previ
0	56	housemaid	married	basic.4y	no	no	no	telephone	may	mon	...	1	999	
1	57	services	married	high.school	unknown	no	no	telephone	may	mon	...	1	999	
2	37	services	married	high.school	no	yes	no	telephone	may	mon	...	1	999	
3	40	admin.	married	basic.6y	no	no	no	telephone	may	mon	...	1	999	
4	56	services	married	high.school	no	no	yes	telephone	may	mon	...	1	999	
5	45	services	married	basic.9y	unknown	no	no	telephone	may	mon	...	1	999	
6	59	admin.	married	professional.course	no	no	no	telephone	may	mon	...	1	999	
7	41	blue-collar	married	unknown	unknown	no	no	telephone	may	mon	...	1	999	
8	24	technician	single	professional.course	no	yes	no	telephone	may	mon	...	1	999	
9	25	services	single	high.school	no	yes	no	telephone	may	mon	...	1	999	
10	41	blue-collar	married	unknown	unknown	no	no	telephone	may	mon	...	1	999	
11	25	services	single	high.school	no	yes	no	telephone	may	mon	...	1	999	
12	29	blue-collar	single	high.school	no	no	yes	telephone	may	mon	...	1	999	
13	57	housemaid	divorced	basic.4y	no	yes	no	telephone	may	mon	...	1	999	
14	35	blue-collar	married	basic.6y	no	yes	no	telephone	may	mon	...	1	999	
15	54	retired	married	basic.9y	unknown	yes	yes	telephone	may	mon	...	1	999	
16	35	blue-collar	married	basic.6y	no	yes	no	telephone	may	mon	...	1	999	
17	46	blue-collar	married	basic.6y	unknown	yes	yes	telephone	may	mon	...	1	999	
18	50	blue-collar	married	basic.9y	no	yes	yes	telephone	may	mon	...	1	999	
19	39	management	single	basic.9y	unknown	no	no	telephone	may	mon	...	1	999	

20 rows × 21 columns



Step 3: Exploratory Data Analysis

An exploratory data analysis (EDA) was conducted to examine the structure, quality, and characteristics of the dataset prior to applying any preprocessing methods or developing machine learning models.

Step 3.1: — Dataset structure and feature types

The first stage of the analysis revealed 41,188 observations and 21 variables presented as; 10 numerical and 11 categorical attributes, including the target variable y.

In [4]:
```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Feature type counts:")
print(df.dtypes.value_counts())

numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()

print("Numerical features:", numerical_cols)
print("Categorical features:", categorical_cols)
```

```
Rows: 41188
Columns: 21
Feature type counts:
object     11
int64       5
float64     5
Name: count, dtype: int64
Numerical features: ['age', 'duration', 'campaign', 'pdays', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed']
Categorical features: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome', 'y']
```

Step 3.2: Data Summary and Categorical Values

Descriptive analysis were conducted on both categorical and numerical variables to identify their feature distributions, and value ranges. Additionally, categorical variables were inspected to examine the frequency distribution of each category.

In [5]:
```
print(df[numerical_cols].describe())

for col in categorical_cols:
    print(col)
    print(df[col].value_counts())
```

```
                age      duration      campaign         pdays      previous  \
count  41188.00000  41188.000000  41188.000000  41188.000000  41188.000000
mean      40.02406    258.285010      2.567593    962.475454      0.172963
std       10.42125    259.279249      2.770014    186.910907      0.494901
min       17.00000      0.000000      1.000000      0.000000      0.000000
25%       32.00000    102.000000      1.000000    999.000000      0.000000
50%       38.00000    180.000000      2.000000    999.000000      0.000000
75%       47.00000    319.000000      3.000000    999.000000      0.000000
max       98.00000   4918.000000     56.000000    999.000000      7.000000

       emp.var.rate  cons.price.idx  cons.conf.idx     euribor3m   nr.employed
count  41188.000000    41188.000000   41188.000000  41188.000000  41188.000000
mean       0.081886       93.575664     -40.502600      3.621291   5167.035911
std        1.570960        0.578840       4.628198      1.734447     72.251528
min       -3.400000       92.201000     -50.800000      0.634000   4963.600000
25%       -1.800000       93.075000     -42.700000      1.344000   5099.100000
50%        1.100000       93.749000     -41.800000      4.857000   5191.000000
75%        1.400000       93.994000     -36.400000      4.961000   5228.100000
max        1.400000       94.767000     -26.900000      5.045000   5228.100000
job
job
admin.           10422
blue-collar       9254
technician        6743
services          3969
management        2924
retired           1720
entrepreneur      1456
self-employed     1421
housemaid         1060
unemployed        1014
student            875
unknown            330
Name: count, dtype: int64
marital
marital
married     24928
single      11568
divorced     4612
unknown        80
Name: count, dtype: int64
```

```
education
education
university.degree      12168
high.school             9515
basic.9y                6045
professional.course     5243
basic.4y                4176
basic.6y                2292
unknown                 1731
illiterate                18
Name: count, dtype: int64
default
default
no         32588
unknown     8597
yes            3
Name: count, dtype: int64
housing
housing
yes        21576
no         18622
unknown      990
Name: count, dtype: int64
loan
loan
no         33950
yes         6248
unknown      990
Name: count, dtype: int64
contact
contact
cellular     26144
telephone    15044
Name: count, dtype: int64
month
month
may    13769
jul     7174
aug     6178
jun     5318
nov     4101
```

```
apr     2632
oct      718
sep      570
mar      546
dec      182
Name: count, dtype: int64
day_of_week
day_of_week
thu    8623
mon    8514
wed    8134
tue    8090
fri    7827
Name: count, dtype: int64
poutcome
poutcome
nonexistent    35563
failure         4252
success         1373
Name: count, dtype: int64
y
y
no     36548
yes     4640
Name: count, dtype: int64
```

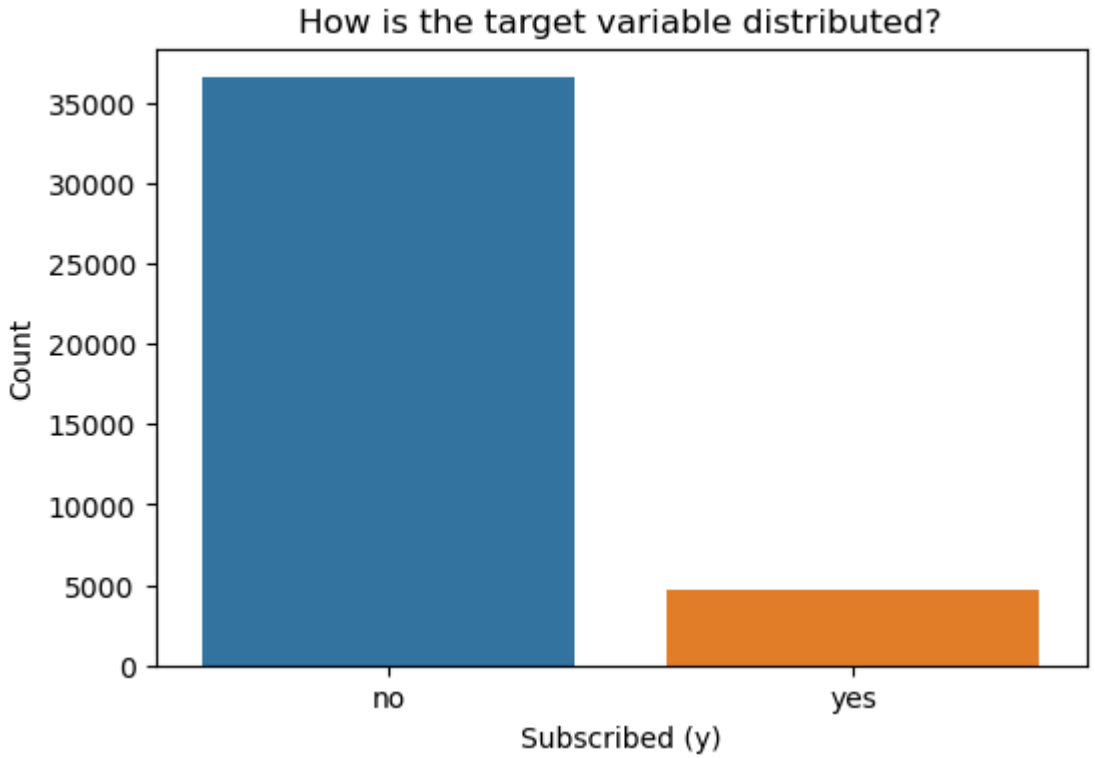
Step 3.3: Target Distribution

The target variable was investigated to assess the distribution of customers who subscribed and those who did not subscribe.

In [6]:
```
print(df["y"].value_counts())
print((df["y"].value_counts(normalize=True) * 100).round(2))

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="y", hue="y", legend=False)
plt.title("How is the target variable distributed?")
plt.xlabel("Subscribed (y)")
plt.ylabel("Count")
plt.show()
```

```
y
no     36548
yes     4640
Name: count, dtype: int64
y
no     88.73
yes    11.27
Name: proportion, dtype: float64
```



Step 3.4: Data Quality Checks

Data quality checks were conducted to identify potential issues such as duplicate records, missing values & unknown data entries that could negatively impact the machine learning models.

In [7]:
```
print("Number of duplicate rows:", df.duplicated().sum())
```

```python
print("\nMissing values per column:")
print(df.isnull().sum())

print("\n'unknown' values per column:")
unknown_counts = (df == "unknown").sum()
print(unknown_counts[unknown_counts > 0])
```

```
Number of duplicate rows: 12

Missing values per column:
age               0
job               0
marital           0
education         0
default           0
housing           0
loan              0
contact           0
month             0
day_of_week       0
duration          0
campaign          0
pdays             0
previous          0
poutcome          0
emp.var.rate      0
cons.price.idx    0
cons.conf.idx     0
euribor3m         0
nr.employed       0
y                 0
dtype: int64

'unknown' values per column:
job           330
marital        80
education    1731
default      8597
housing       990
loan          990
dtype: int64
```

Step 3.5: Numerical Distributions and Outliers

The numerical variables were examined using histograms and boxplots. Histograms showed the shape and skewness of each distribution, while boxplots highlighted potential outliers. This assessment informed the outlier treatment, feature scaling, and feature engineering applied during preprocessing.

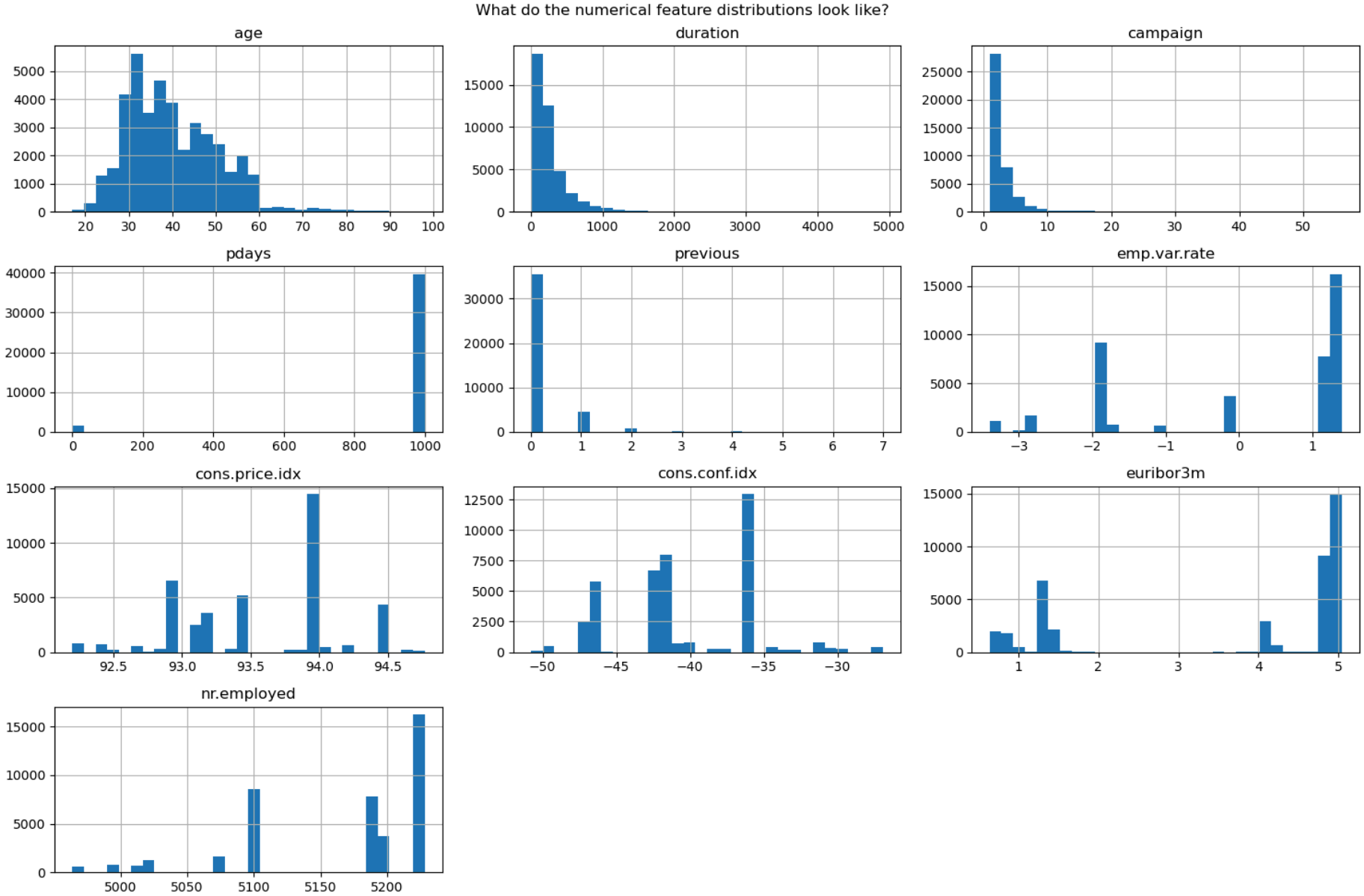
In [8]:
```
df[numerical_cols].hist(bins=30, figsize=(15, 10))
plt.suptitle("What do the numerical feature distributions look like?")
plt.tight_layout()
plt.show()

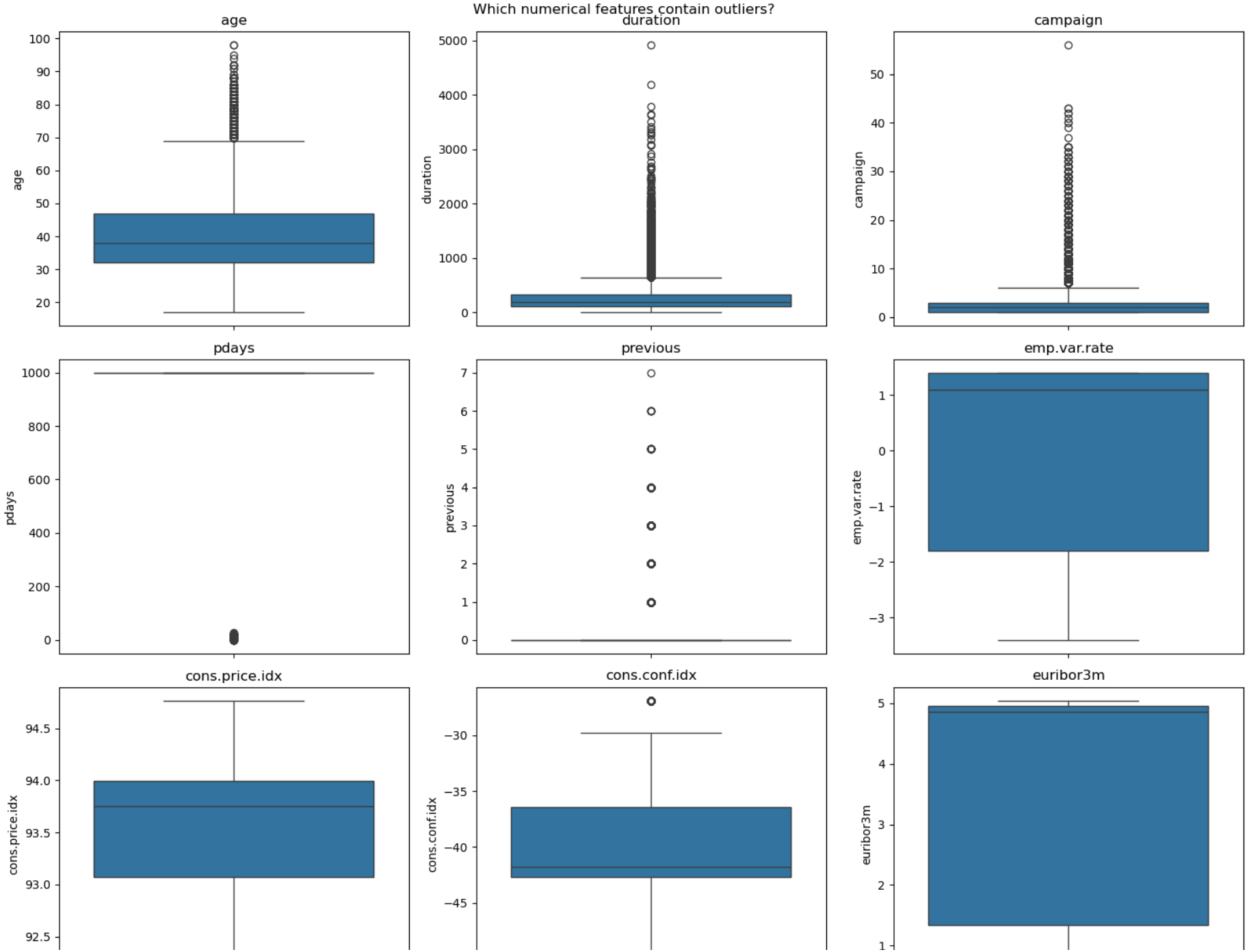
n_cols = 3
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows))
axes = axes.flatten()

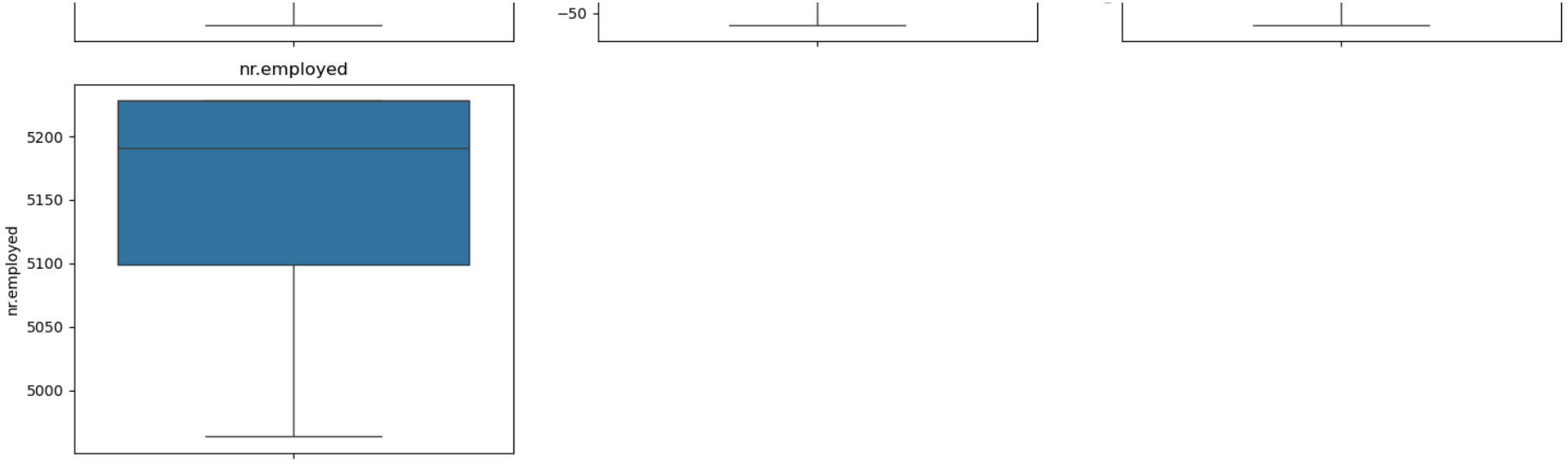
for i, col in enumerate(numerical_cols):
    sns.boxplot(data=df, y=col, ax=axes[i])
    axes[i].set_title(col)

for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.suptitle("Which numerical features contain outliers?")
plt.tight_layout()
plt.show()
```


What do the numerical feature distributions look like?
age
duration
campaign
pdays
previous
emp.var.rate
cons.price.idx
cons.conf.idx
euribor3m
nr.employed

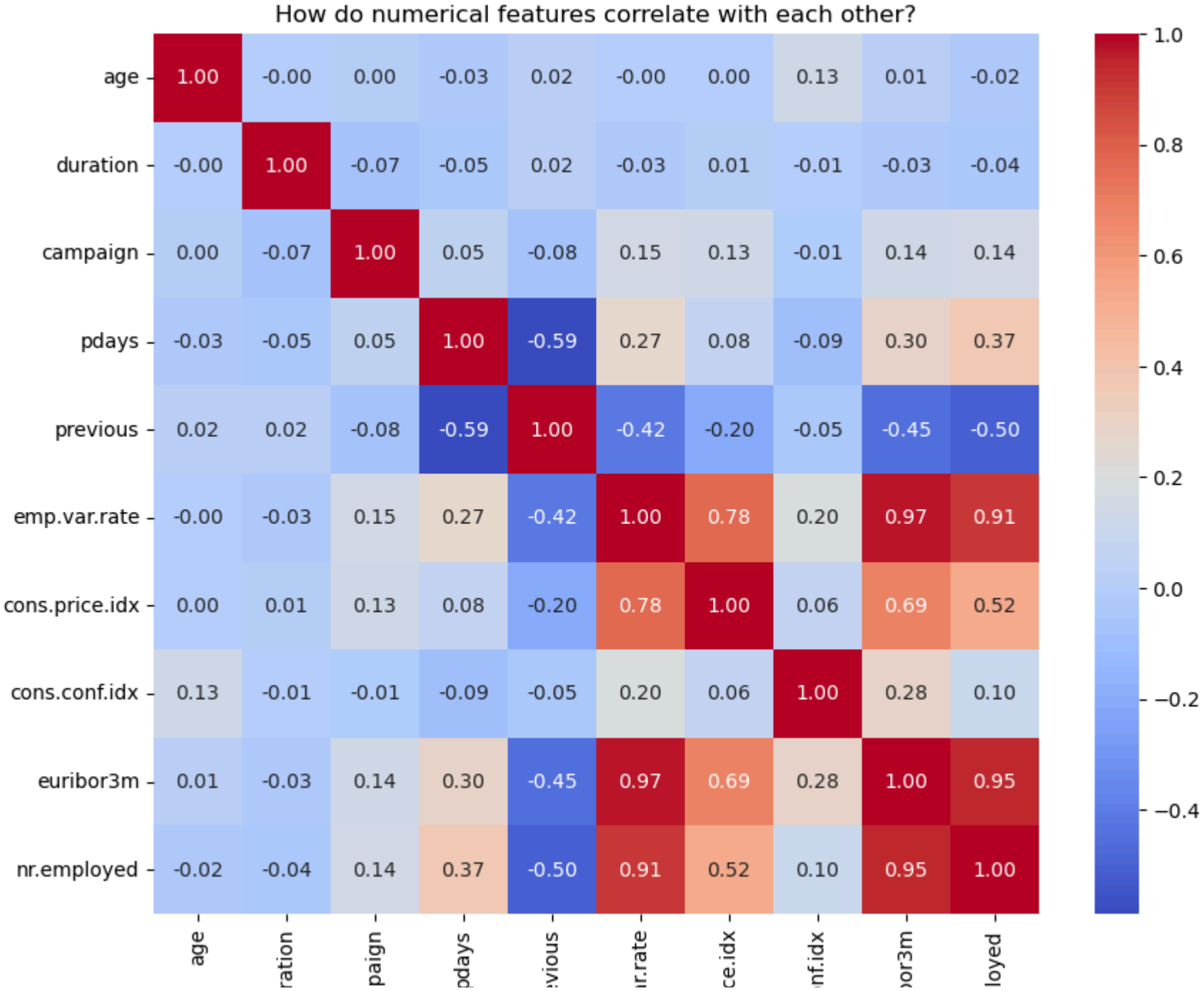

Which numerical features contain outliers?
age
duration
campaign
pdays
previous
emp.var.rate
cons.price.idx
cons.conf.idx
euribor3m



Step 3.6 — Correlations Between Numerical Features

Correlation analysis was conducted to identify strongly correlated features that signals a potential multicollinearity relationship among the predictor variables, particularly for algorithms sensitive to correlated features such as Logistic Regression. The analysis also provided an understanding of relationships between numerical variables before preprocessing and model development.

In [9]:
```
plt.figure(figsize=(10, 8))
sns.heatmap(df[numerical_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("How do numerical features correlate with each other?")
plt.show()
```


How do numerical features correlate with each other?
age 1.00 -0.00 0.00 -0.03 0.02 -0.00 0.00 0.13 0.01 -0.02
duration -0.00 1.00 -0.07 -0.05 0.02 -0.03 0.01 -0.01 -0.03 -0.04
campaign 0.00 -0.07 1.00 0.05 -0.08 0.15 0.13 -0.01 0.14 0.14
pdays -0.03 -0.05 0.05 1.00 -0.59 0.27 0.08 -0.09 0.30 0.37
previous 0.02 0.02 -0.08 -0.59 1.00 -0.42 -0.20 -0.05 -0.45 -0.50
emp.var.rate -0.00 -0.03 0.15 0.27 -0.42 1.00 0.78 0.20 0.97 0.91
cons.price.idx 0.00 0.01 0.13 0.08 -0.20 0.78 1.00 0.06 0.69 0.52
cons.conf.idx 0.13 -0.01 -0.01 -0.09 -0.05 0.20 0.06 1.00 0.28 0.10
euribor3m 0.01 -0.03 0.14 0.30 -0.45 0.97 0.69 0.28 1.00 0.95
nr.employed -0.02 -0.04 0.14 0.37 -0.50 0.91 0.52 0.10 0.95 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4



Step 3.7: Feature Relationships with Subscription Outcome

Numerical variables were compared across the target classes using grouped boxplots, while subscription rates were calculated for each category of the categorical variables. This provided an initial overview of how the predictors relate to subscription before feature engineering and model development applied.

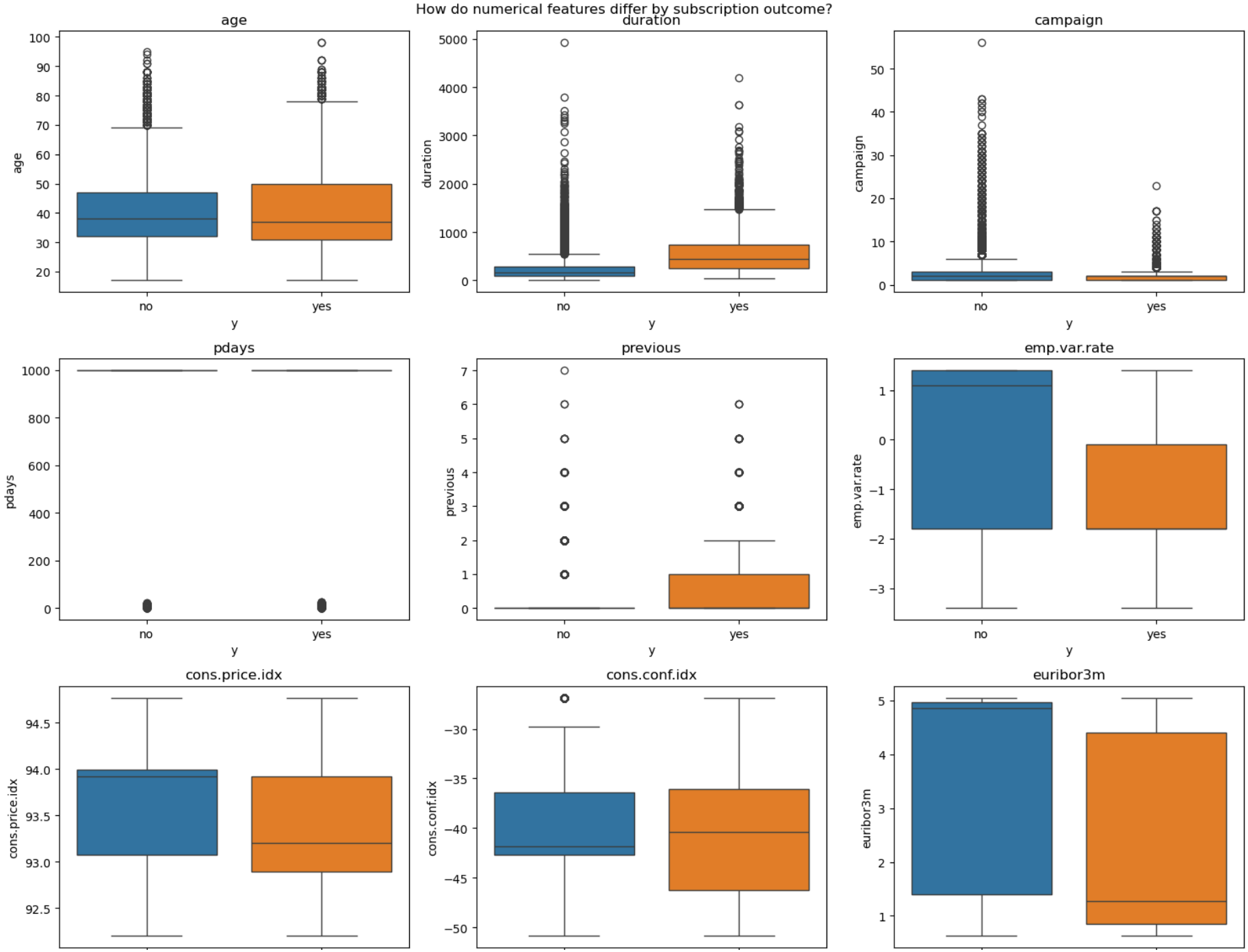
In [10]:
```
# Numerical features by subscription outcome
n_cols = 3
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows))
axes = axes.flatten()

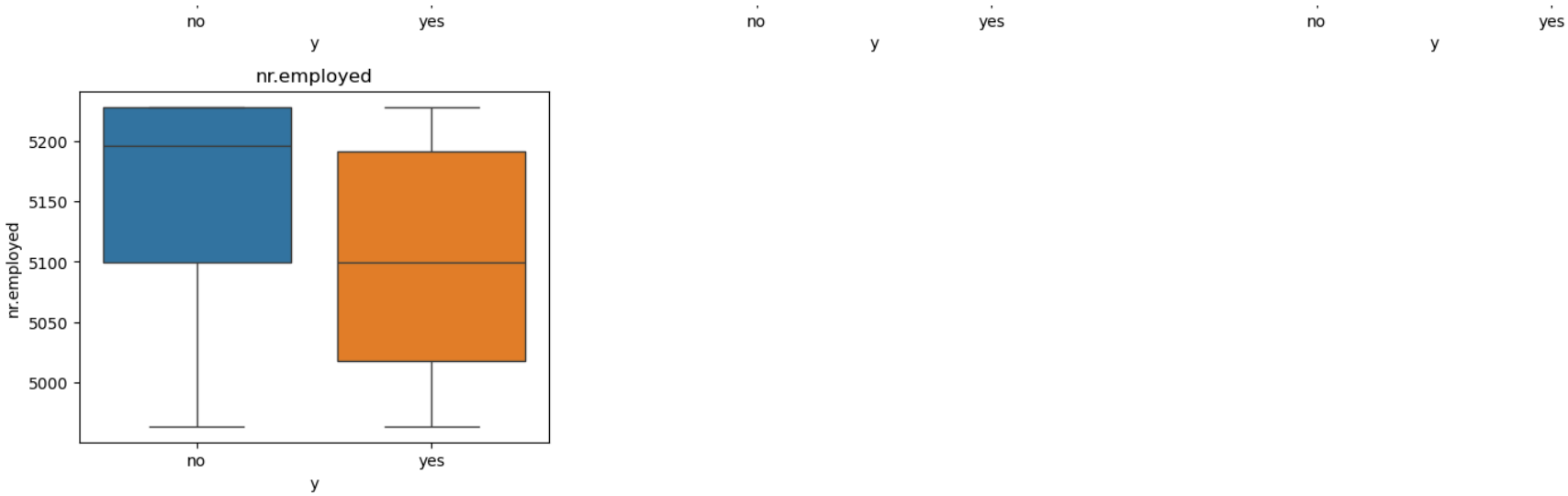
for i, col in enumerate(numerical_cols):
    sns.boxplot(data=df, x="y", y=col, hue="y", legend=False, ax=axes[i])
    axes[i].set_title(col)

for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.suptitle("How do numerical features differ by subscription outcome?")
plt.tight_layout()
plt.show()

# Categorical features — subscription rate per category
for col in categorical_cols:
    if col == "y":
        continue
    rate = df.groupby(col)["y"].apply(lambda x: (x == "yes").mean() * 100).sort_values(ascending=False)
    print("Subscription rate (%) by", col)
    print(rate.round(2))
```


How do numerical features differ by subscription outcome?
age
duration
campaign
pdays
previous
emp.var.rate
cons.price.idx
cons.conf.idx
euribor3m
no
yes
y


no
yes
y
no
yes
y
no
yes
y
nr.employed
5200
5150
5100
5050
5000
nr.employed
no
yes
y

```
Subscription rate (%) by job
job
student          31.43
retired          25.23
unemployed       14.20
admin.           12.97
management       11.22
unknown          11.21
technician       10.83
self-employed    10.49
housemaid        10.00
entrepreneur      8.52
services          8.14
blue-collar       6.89
Name: y, dtype: float64
Subscription rate (%) by marital
marital
unknown     15.00
single      14.00
divorced    10.32
married     10.16
Name: y, dtype: float64
Subscription rate (%) by education
education
illiterate             22.22
unknown                14.50
university.degree      13.72
professional.course    11.35
high.school            10.84
basic.4y               10.25
basic.6y                8.20
basic.9y                7.82
Name: y, dtype: float64
Subscription rate (%) by default
default
no         12.88
unknown     5.15
yes         0.00
Name: y, dtype: float64
Subscription rate (%) by housing
housing
```

```
yes        11.62
no         10.88
unknown    10.81
Name: y, dtype: float64
Subscription rate (%) by loan
loan
no         11.34
yes        10.93
unknown    10.81
Name: y, dtype: float64
Subscription rate (%) by contact
contact
cellular     14.74
telephone     5.23
Name: y, dtype: float64
Subscription rate (%) by month
month
mar    50.55
dec    48.90
sep    44.91
oct    43.87
apr    20.48
aug    10.60
jun    10.51
nov    10.14
jul     9.05
may     6.43
Name: y, dtype: float64
Subscription rate (%) by day_of_week
day_of_week
thu    12.12
tue    11.78
wed    11.67
fri    10.81
mon     9.95
Name: y, dtype: float64
Subscription rate (%) by poutcome
poutcome
success        65.11
failure        14.23
```

```
nonexistent     8.83
Name: y, dtype: float64
```

Step 4: Data Preprocessing

Step 4.1: Copy of original dataset

A copy of the original dataset was made to preserve the critical data in case machine learning pipeline experiences faults.

In [11]:
```
df_raw = df.copy()
df_clean = df.copy()
```

Step 4.2: Remove Duplicates

The first preprocessing stage consisted of removing 12 duplicate observations, resulting in 41.176 records. Removal of duplicates is crucial implementation as unresolved data-quality issues may negatively impact the quality of subsequent analyses.

In [12]:
```
rows_before = df_clean.shape[0]
df_clean = df_clean.drop_duplicates()
df_clean = df_clean.reset_index(drop=True)
rows_after = df_clean.shape[0]

print("Rows before:", rows_before)
print("Rows after:", rows_after)
print("Duplicates removed:", rows_before - rows_after)
```

```
Rows before: 41188
Rows after: 41176
Duplicates removed: 12
```

Step 4.3: Drop Duration (data leakage)

The **duration** variable was dropped because **call duration** only being known after the contact has been made, resulting in 20 variables remaining. Including it would introduce data leakage due to **call duration** info being unavailable when customers are selected for future contact.

In [13]:
```
# duration is only known after a call has ended
# including it causes data leakage and inflates model performance
if 'duration' in df_clean.columns:
    df_clean = df_clean.drop(columns=['duration'])

print("Shape after removal:", df_clean.shape)
print("'duration' in columns:", 'duration' in df_clean.columns)
```

```
Shape after removal: (41176, 20)
'duration' in columns: False
```

Step 4.4: Feature Engineering: pdays → previously_contacted

A customer with **pdays** value of 999 indicates that the customer was not previously contacted (Moro, Rita and Cortez, 2014b). This information informed the creation of a binary variable, **previously_contacted**, after which the original **pdays** variable was dropped. The target was encoded to 0 when the customer did not subscribe and 1 when the customer subscribed, resulting in 19 predictors and one binary target.

In [14]:
```
df_clean['previously_contacted'] = (df_clean['pdays'] != 999).astype(int)
df_clean = df_clean.drop(columns=['pdays'])

print(df_clean['previously_contacted'].value_counts())
print("'pdays' in columns:", 'pdays' in df_clean.columns)
```

```
previously_contacted
0    39661
1     1515
Name: count, dtype: int64
'pdays' in columns: False
```

Step 4.5: Target Variable Encoding

The target variable (y) was converted into a binary numerical format, with subscribers encoded as 1 and non-subscribers encoded as 0, making it compatible with the selected machine learning algorithms.

In [15]:
```
print("Before encoding:")
print(df_clean['y'].value_counts())

df_clean['y'] = df_clean['y'].map({'yes': 1, 'no': 0}).astype(int)
```

```
print("\nAfter encoding:")
print(df_clean['y'].value_counts())

print("\nClass proportions:")
print(df_clean['y'].value_counts(normalize=True))

print("\nMissing values in y:", df_clean['y'].isnull().sum())
```

```
Before encoding:
y
no     36537
yes     4639
Name: count, dtype: int64

After encoding:
y
0    36537
1     4639
Name: count, dtype: int64

Class proportions:
y
0    0.887337
1    0.112663
Name: proportion, dtype: float64

Missing values in y: 0
```

Step 4.6: Separate Predictors and Target Variable

The dataset was separated into predictor variables (X) and the encoded target variable (y), establishing the standard structure required for supervised machine learning.

In [16]:
```
X = df_clean.drop(columns=['y'])
y = df_clean['y']

print("X shape:", X.shape)
print("y shape:", y.shape)
print("\nPredictor columns:", list(X.columns))
```

```
dummy_check = [col for col in X.columns if '_' in col and X[col].nunique() == 2]
print("\nPossible existing dummy variables:", dummy_check)
```

```
X shape: (41176, 19)
y shape: (41176,)

Predictor columns: ['age', 'job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'cam
paign', 'previous', 'poutcome', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed', 'previously_cont
acted']

Possible existing dummy variables: ['previously_contacted']
```

Step 4.7: Train-test-validation split (60/20/20, stratified)

The transformed dataset was split into training, validation and test sets of 60%, 20% and 20%. The training set contained 24.705 observations, the validation set had 8.235, and the test set consisting of 8.236 records. Approximately 11.27% of the positive-class was maintained in each subset after stratification was implemented.

In [18]:
```
from sklearn.model_selection import train_test_split

X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
X_train, X_validation, y_train, y_validation = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
)

X_train = X_train.copy()
X_validation = X_validation.copy()
X_test = X_test.copy()

print("X_train:", X_train.shape)
print("X_validation:", X_validation.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_validation:", y_validation.shape)
print("y_test:", y_test.shape)
```

```
X_train: (24705, 19)
X_validation: (8235, 19)
X_test: (8236, 19)
y_train: (24705,)
y_validation: (8235,)
y_test: (8236,)
```

Step 4.8: Validate train-test-validation Split

Further validation checks were carried out to ensure that was no overlap between the three subsets.

In [19]:
```
total = len(X)
train_pct = len(X_train) / total
val_pct = len(X_validation) / total
test_pct = len(X_test) / total

print("Training proportion:", round(train_pct, 3))
print("Validation proportion:", round(val_pct, 3))
print("Test proportion:", round(test_pct, 3))
print("Total proportion:", round(train_pct + val_pct + test_pct, 3))

print("\nClass balance (train):", y_train.mean())
print("Class balance (validation):", y_validation.mean())
print("Class balance (test):", y_test.mean())

train_idx = set(X_train.index)
val_idx = set(X_validation.index)
test_idx = set(X_test.index)

assert train_idx.isdisjoint(val_idx)
assert train_idx.isdisjoint(test_idx)
assert val_idx.isdisjoint(test_idx)
assert len(X_train) + len(X_validation) + len(X_test) == len(X)

print("\nAll checks passed: no overlap, all records accounted for.")
```

```
Training proportion: 0.6
Validation proportion: 0.2
Test proportion: 0.2
Total proportion: 1.0

Class balance (train): 0.11264926128314107
Class balance (validation): 0.11268973891924712
Class balance (test): 0.11267605633802817

All checks passed: no overlap, all records accounted for.
```

Step 4.9: Identify Feature Types

Both categorical and numerical feature type was identified to distinguish between the data type, which informed the preprocessing decision prior to applying any algorithm-specific preprocessing technique.

In [20]:
```python
numerical_features = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X_train.select_dtypes(include=['object']).columns.tolist()

print("Numerical features:", numerical_features)
print("Number of numerical features:", len(numerical_features))
print("\nCategorical features:", categorical_features)
print("Number of categorical features:", len(categorical_features))

assert len(categorical_features) > 0, "No categorical features found"
```

```
Numerical features: ['age', 'campaign', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employe
d', 'previously_contacted']
Number of numerical features: 9

Categorical features: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcom
e']
Number of categorical features: 10
```

Step 4.10: Outlier Capping (IQR method — fit on train, applied to train and test)

Outlier capping was implemented using interquartile-range boundaries learned from the training data. Capping extreme records instead of deleting them preserves all customer records, which is beneficial method when extreme observations might be valid observation.

In [21]:
```
outlier_bounds = {}
for col in ['age', 'campaign']:
    Q1 = X_train[col].quantile(0.25)
    Q3 = X_train[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_bounds[col] = (lower, upper)

print("Learned bounds:", outlier_bounds)

for col, (lower, upper) in outlier_bounds.items():
    X_train[col] = X_train[col].clip(lower, upper)
    X_validation[col] = X_validation[col].clip(lower, upper)
    X_test[col] = X_test[col].clip(lower, upper)

print("\nSummary after clipping (train):")
print(X_train[['age', 'campaign']].describe())
```

```
Learned bounds: {'age': (np.float64(9.5), np.float64(69.5)), 'campaign': (np.float64(-2.0), np.float64(6.0))}

Summary after clipping (train):
                age      campaign
count  24705.000000  24705.000000
mean      39.935580      2.275046
std       10.163631      1.552905
min       17.000000      1.000000
25%       32.000000      1.000000
50%       38.000000      2.000000
75%       47.000000      3.000000
max       69.500000      6.000000
```

Step 4.11: One-Hot Encoding (Branch A: Logistic Regression)

One-hot encoding was implemented on the categorical features, transforming them into binary columns of 1 and 0 that allow models such as Logistic Regression to process the categories without assuming any mathematical order or hierarchy. This resulted in 52 features in total.

In [22]:
```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
encoder.fit(X_train[categorical_features])
encoded_cols = encoder.get_feature_names_out(categorical_features)

X_train_cat = pd.DataFrame(encoder.transform(X_train[categorical_features]), columns=encoded_cols, index=X_train.index)
X_validation_cat = pd.DataFrame(encoder.transform(X_validation[categorical_features]), columns=encoded_cols, index=X_validatio
X_test_cat = pd.DataFrame(encoder.transform(X_test[categorical_features]), columns=encoded_cols, index=X_test.index)

X_train_processed = pd.concat([X_train[numerical_features], X_train_cat], axis=1)
X_validation_processed = pd.concat([X_validation[numerical_features], X_validation_cat], axis=1)
X_test_processed = pd.concat([X_test[numerical_features], X_test_cat], axis=1)

print("X_train_processed:", X_train_processed.shape)
print("X_validation_processed:", X_validation_processed.shape)
print("X_test_processed:", X_test_processed.shape)
print("\nExample encoded feature names:", list(encoded_cols[:5]))

assert X_train_processed.isnull().sum().sum() == 0
assert list(X_train_processed.columns) == list(X_validation_processed.columns) == list(X_test_processed.columns)
assert X_train_processed.shape[0] == y_train.shape[0]
print("\nAll checks passed.")
```

```
X_train_processed: (24705, 52)
X_validation_processed: (8235, 52)
X_test_processed: (8236, 52)

Example encoded feature names: ['job_blue-collar', 'job_entrepreneur', 'job_housemaid', 'job_management', 'job_retired']

All checks passed.
```

Step 4.12: Feature Scaling (Branch A: Logistic Regression)

Additionally, all features were standardised to a comparable scale when using Logistic Regression, with the scaler fitted on the training data only.

In [23]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train_processed)
```

```
X_train_lr = pd.DataFrame(
    scaler.transform(X_train_processed),
    columns=X_train_processed.columns,
    index=X_train_processed.index
)
X_validation_lr = pd.DataFrame(
    scaler.transform(X_validation_processed),
    columns=X_validation_processed.columns,
    index=X_validation_processed.index
)
X_test_lr = pd.DataFrame(
    scaler.transform(X_test_processed),
    columns=X_test_processed.columns,
    index=X_test_processed.index
)

print("X_train_lr:", X_train_lr.shape)
print("X_validation_lr:", X_validation_lr.shape)
print("X_test_lr:", X_test_lr.shape)
```

```
X_train_lr: (24705, 52)
X_validation_lr: (8235, 52)
X_test_lr: (8236, 52)
```

Step 4.13: Ordinal Encoding (Branch B: Random Forest and XGBoost)

Ordinal encoding was implemented on the categorical features, converting each category into an integer value for the tree-based models, with the encoder fitted on the training data only. This created a consistent numerical representation, but may impose an artificial ordering on nominal categories.

In [24]:
```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
ordinal_encoder.fit(X_train[categorical_features])

X_train_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_train[categorical_features]),
    columns=categorical_features,
    index=X_train.index
)
```

```
X_validation_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_validation[categorical_features]),
    columns=categorical_features,
    index=X_validation.index
)
X_test_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_test[categorical_features]),
    columns=categorical_features,
    index=X_test.index
)

X_train_tree = pd.concat([X_train[numerical_features], X_train_cat_ord], axis=1)
X_validation_tree = pd.concat([X_validation[numerical_features], X_validation_cat_ord], axis=1)
X_test_tree = pd.concat([X_test[numerical_features], X_test_cat_ord], axis=1)

print("X_train_tree:", X_train_tree.shape)
print("X_validation_tree:", X_validation_tree.shape)
print("X_test_tree:", X_test_tree.shape)
```

```
X_train_tree: (24705, 19)
X_validation_tree: (8235, 19)
X_test_tree: (8236, 19)
```

Step 4.14: Final Preprocessing Validation

The final preprocessing stage consisted of checking that there were no missing values and no mismatched row counts before developing, evaluating and interpreting the machine-learning models.

In [25]:
```
print("Missing values - LR branch:")
print(X_train_lr.isnull().sum().sum(), X_validation_lr.isnull().sum().sum(), X_test_lr.isnull().sum().sum())

print("\nMissing values - Tree branch:")
print(X_train_tree.isnull().sum().sum(), X_validation_tree.isnull().sum().sum(), X_test_tree.isnull().sum().sum())

print("\nLR columns match:", list(X_train_lr.columns) == list(X_validation_lr.columns) == list(X_test_lr.columns))
print("Tree columns match:", list(X_train_tree.columns) == list(X_validation_tree.columns) == list(X_test_tree.columns))

print("\nRow counts match y - LR:")
print(X_train_lr.shape[0] == y_train.shape[0])
print(X_validation_lr.shape[0] == y_validation.shape[0])
```

```
print(X_test_lr.shape[0] == y_test.shape[0])

print("\nRow counts match y - Tree:")
print(X_train_tree.shape[0] == y_train.shape[0])
print(X_validation_tree.shape[0] == y_validation.shape[0])
print(X_test_tree.shape[0] == y_test.shape[0])
```

```
Missing values - LR branch:
0 0 0

Missing values - Tree branch:
0 0 0

LR columns match: True
Tree columns match: True

Row counts match y - LR:
True
True
True

Row counts match y - Tree:
True
True
True
```

Machine Learning Algorithm Implementation.

A common experimental setup was used to develop Logistic Regression, Random Forest and XGBoost. Logistic Regression provided an interpretable baseline. Random Forest, an ensemble of multiple decision trees, was used to capture nonlinear patterns and reduce the instability of an individual tree. XGBoost, a regularised, sequential gradient-boosted tree algorithm, was implemented to learn more complex relationships.

All three models were cross-validated, and their hyperparameters were optimised using 5-fold cross-validation and GridSearchCV. This strategy enabled a systematic comparison of candidate configurations and reduced reliance on a single training split.

Step 5: Logistic Regression

Step 5.1: Logistic Regression 5-fold Cross-Validation

In [36]:
```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

lr_pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', LogisticRegression(random_state=42, max_iter=1000))
])

cv_results = cross_validate(lr_pipeline, X_train_lr, y_train, cv=cv, scoring=scoring)

print("LR (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

```
LR (SMOTE pipeline) 5-fold CV
accuracy : 0.8254 +/- 0.0051
precision : 0.3461 +/- 0.0102
recall : 0.618 +/- 0.0219
f1 : 0.4436 +/- 0.012
roc_auc : 0.7821 +/- 0.0069
```

Step 5.2: GridSearchCV

In [42]:
```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'classifier__C': [0.001, 0.005, 0.01, 0.1, 1, 10],
    'classifier__penalty': ['l1', 'l2'],
    'classifier__solver': ['liblinear']
}
```

```
grid_search = GridSearchCV(lr_pipeline, param_grid, cv=cv, scoring='f1', n_jobs=-1)
grid_search.fit(X_train_lr, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best CV F1:", grid_search.best_score_)

evaluate(grid_search.best_estimator_, X_validation_lr, y_validation, "Validation set (chosen model)")
```

```
Best parameters: {'classifier__C': 10, 'classifier__penalty': 'l2', 'classifier__solver': 'liblinear'}
Best CV F1: 0.44391443794227065
Validation set (chosen model)
Accuracy: 0.82829386763813
Precision: 0.35290556900726394
Recall: 0.6282327586206896
F1: 0.45193798449612405
ROC-AUC: 0.800315474533159
```

Step 5.4: Applying Optimal Parameters Obtained From GridSearchCV

In [43]:
```
lr_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', LogisticRegression(
        C=10, penalty='l2', solver='liblinear',
        random_state=42, max_iter=1000
    ))
])
lr_final.fit(X_train_lr, y_train)

evaluate(lr_final, X_train_lr, y_train, "Training set (final model)")
evaluate(lr_final, X_validation_lr, y_validation, "Validation set (final model)")
```

```
Training set (final model)
Accuracy: 0.8261485529245092
Precision: 0.3474576271186441
Recall: 0.6187567373338124
F1: 0.44501873627083605
ROC-AUC: 0.7883481426963654

Validation set (final model)
Accuracy: 0.82829386763813
Precision: 0.35290556900726394
Recall: 0.6282327586206896
F1: 0.45193798449612405
ROC-AUC: 0.800315474533159
```

Step 5.5: Final evaluation — test set (LR frozen)

In [45]:

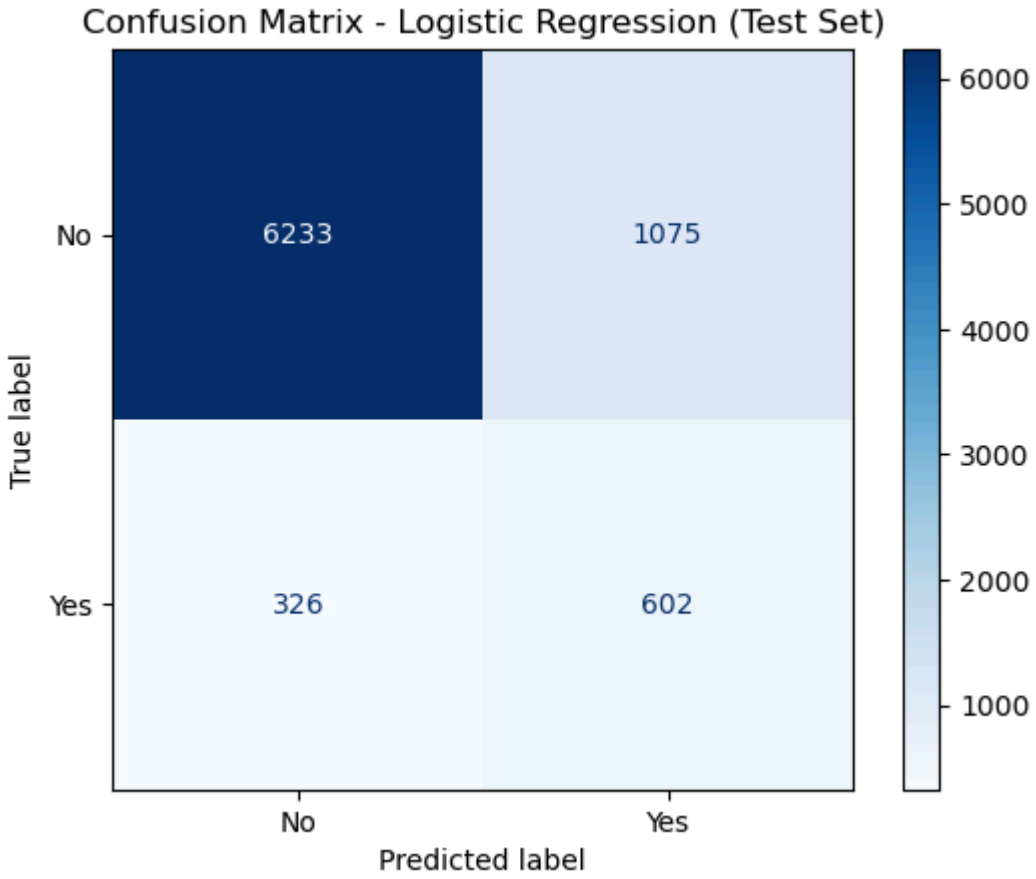
```python
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

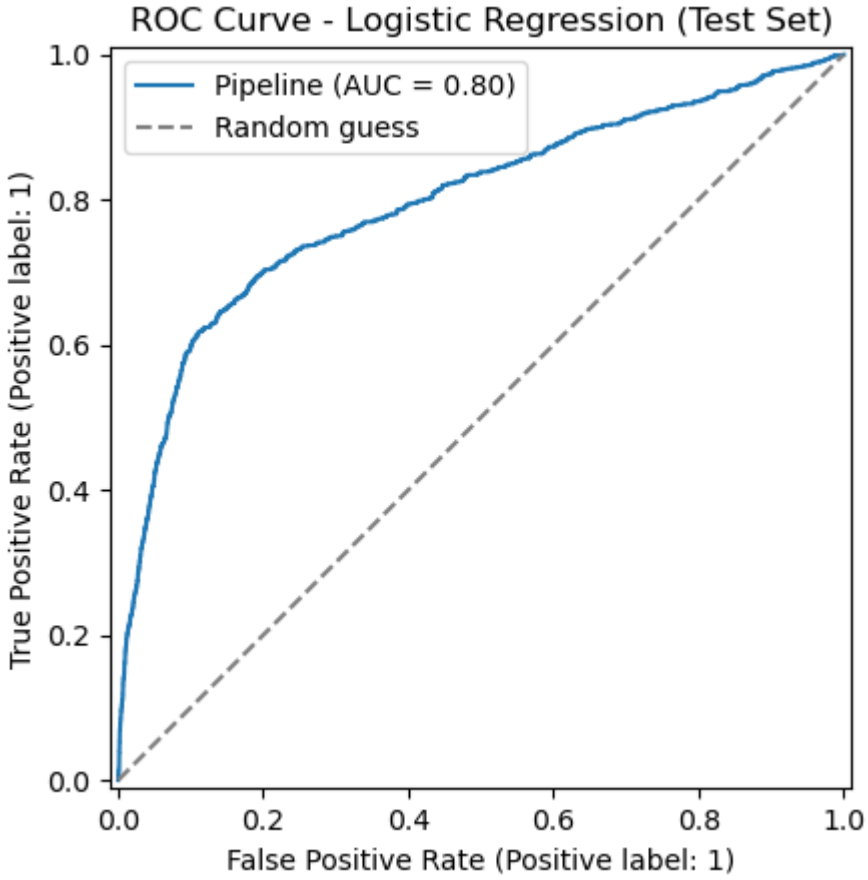
evaluate(lr_final, X_test_lr, y_test, "Test set (final LR evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(lr_final, X_test_lr, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - Logistic Regression (Test Set)")
plt.show()

# ROC-AUC curve
RocCurveDisplay.from_estimator(lr_final, X_test_lr, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - Logistic Regression (Test Set)")
plt.legend()
plt.show()
```

```
Test set (final LR evaluation)
Accuracy: 0.8298931520155415
Precision: 0.358974358974359
Recall: 0.6487068965517241
F1: 0.4621880998080614
ROC-AUC: 0.7964545378942304
```





Step 6: Random Forest

Step 6.1: Random Forest 5-Fold Cross Validation

In [48]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

rf_pipeline = ImbPipeline([
```

```
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(random_state=42))
])

cv_results = cross_validate(rf_pipeline, X_train_tree, y_train, cv=cv, scoring=scoring)

print("RF (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

```
RF (SMOTE pipeline) 5-fold CV
accuracy : 0.8899 +/- 0.0035
precision : 0.5166 +/- 0.0226
recall : 0.345 +/- 0.0196
f1 : 0.4136 +/- 0.0205
roc_auc : 0.7703 +/- 0.0183
```

Step 6.2: GridSearch Hyperparameter Tuning

In [49]:

```
param_grid_rf = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5],
    'classifier__min_samples_leaf': [1, 2]
}

grid_search_rf = GridSearchCV(rf_pipeline, param_grid_rf, cv=cv, scoring='f1', n_jobs=-1)
grid_search_rf.fit(X_train_tree, y_train)

print("Best parameters:", grid_search_rf.best_params_)
print("Best CV F1:", grid_search_rf.best_score_)

evaluate(grid_search_rf.best_estimator_, X_validation_tree, y_validation, "Validation set (chosen model)")
```

```
Best parameters: {'classifier__max_depth': 10, 'classifier__min_samples_leaf': 2, 'classifier__min_samples_split': 5, 'classifi
er__n_estimators': 200}
Best CV F1: 0.4779135067277885
Validation set (chosen model)
Accuracy: 0.8769884638737098
Precision: 0.4608294930875576
Recall: 0.5387931034482759
F1: 0.49677098857426727
ROC-AUC: 0.7991321500875401
```

Step 6.3: Applying Optimal Parameters Obtained From GridSearchCV

In [50]:
```
rf_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(
        n_estimators=200, max_depth=10,
        min_samples_split=5, min_samples_leaf=2,
        random_state=42
    ))
])
rf_final.fit(X_train_tree, y_train)

evaluate(rf_final, X_train_tree, y_train, "Training set (final model)")
evaluate(rf_final, X_validation_tree, y_validation, "Validation set (final model)")
```

```
Training set (final model)
Accuracy: 0.8870269176280106
Precision: 0.4987413467589679
Recall: 0.5695292849443047
F1: 0.5317899681261533
ROC-AUC: 0.8367543218184172

Validation set (final model)
Accuracy: 0.8769884638737098
Precision: 0.4608294930875576
Recall: 0.5387931034482759
F1: 0.49677098857426727
ROC-AUC: 0.7991321500875401
```

Step 6.4: Final evaluation — test set (RF frozen)

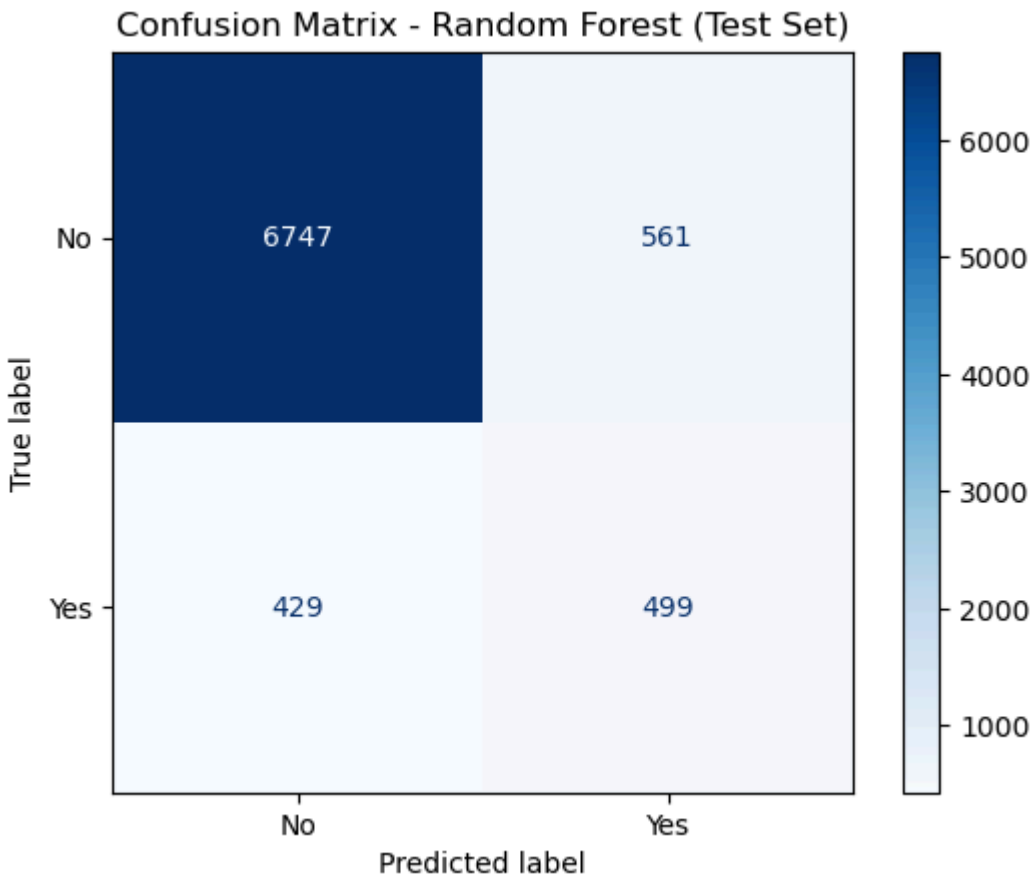
In [51]:
```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

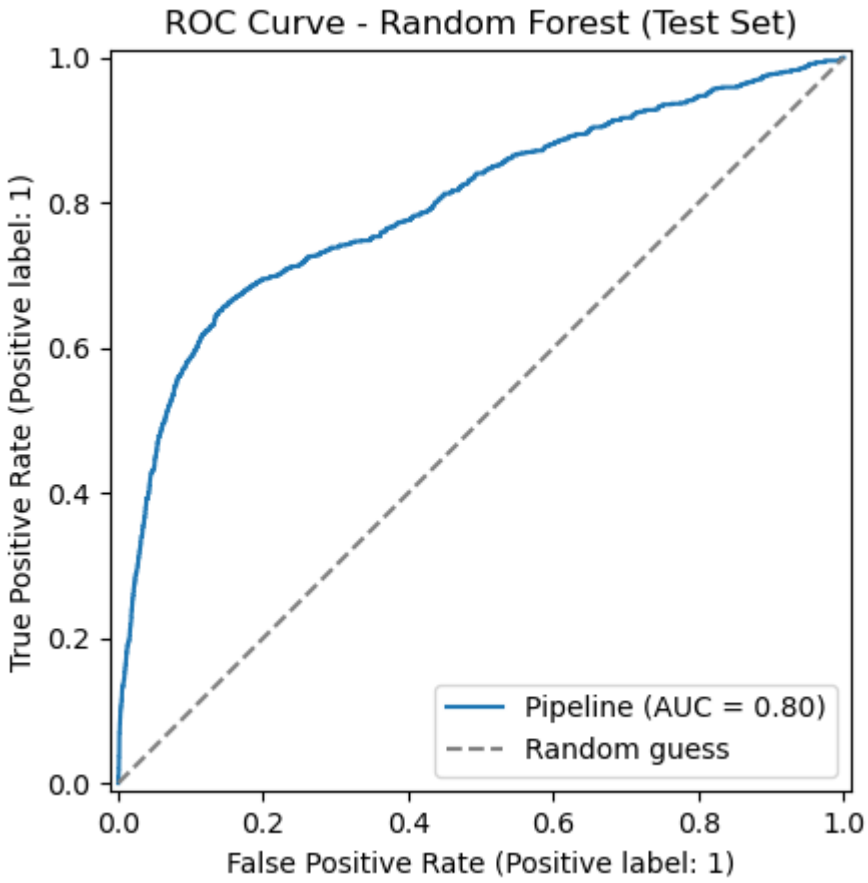
evaluate(rf_final, X_test_tree, y_test, "Test set (final RF evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(rf_final, X_test_tree, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - Random Forest (Test Set)")
plt.show()

# ROC-AUC curve
RocCurveDisplay.from_estimator(rf_final, X_test_tree, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - Random Forest (Test Set)")
plt.legend()
plt.show()
```

```
Test set (final RF evaluation)
Accuracy: 0.8797960174842157
Precision: 0.4707547169811321
Recall: 0.5377155172413793
F1: 0.5020120724346077
ROC-AUC: 0.7976511923635883
```


Confusion Matrix - Random Forest (Test Set)
6747
561
429
499
No
Yes
True label
No
Yes
Predicted label
6000
5000
4000
3000
2000
1000



Step 7: XGBoost Model

Step 7.1: XGBoost 5-fold Cross-Valdidation Model

In [53]:
```
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

xgb_pipeline = ImbPipeline([
```

```
    ('smote', SMOTE(random_state=42)),
    ('classifier', XGBClassifier(random_state=42, eval_metric='logloss'))
])

cv_results = cross_validate(xgb_pipeline, X_train_tree, y_train, cv=cv, scoring=scoring)

print("XGBoost (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

```
XGBoost (SMOTE pipeline) 5-fold CV
accuracy : 0.8954 +/- 0.0035
precision : 0.5676 +/- 0.0296
recall : 0.3033 +/- 0.0185
f1 : 0.395 +/- 0.0199
roc_auc : 0.7675 +/- 0.0099
```

7.2: GridSearch Hyperparameter Tuning

In [54]:

```
param_grid_xgb = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [3, 5, 7],
    'classifier__learning_rate': [0.01, 0.1, 0.2],
    'classifier__subsample': [0.8, 1.0]
}

grid_search_xgb = GridSearchCV(xgb_pipeline, param_grid_xgb, cv=cv, scoring='f1', n_jobs=-1)
grid_search_xgb.fit(X_train_tree, y_train)

print("Best parameters:", grid_search_xgb.best_params_)
print("Best CV F1:", grid_search_xgb.best_score_)

evaluate(grid_search_xgb.best_estimator_, X_validation_tree, y_validation, "Validation set (chosen model)")
```

```
Best parameters: {'classifier__learning_rate': 0.01, 'classifier__max_depth': 5, 'classifier__n_estimators': 200, 'classifier__
subsample': 1.0}
Best CV F1: 0.48143846292357334
Validation set (chosen model)
Accuracy: 0.8763812993321191
Precision: 0.4585635359116022
Recall: 0.5366379310344828
F1: 0.4945382323733863
ROC-AUC: 0.7948993761296441
```

Step 7.3: GridSearchCV Optimal Parameters

In [55]:
```python
xgb_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', XGBClassifier(
        n_estimators=200, max_depth=5,
        learning_rate=0.01, subsample=1.0,
        random_state=42, eval_metric='logloss'
    ))
])
xgb_final.fit(X_train_tree, y_train)

evaluate(xgb_final, X_train_tree, y_train, "Training set (final model)")
evaluate(xgb_final, X_validation_tree, y_validation, "Validation set (final model)")
```

```
Training set (final model)
Accuracy: 0.8754503137016798
Precision: 0.45493562231759654
Recall: 0.5332375134746676
F1: 0.4909842845326716
ROC-AUC: 0.7883556661856332

Validation set (final model)
Accuracy: 0.8763812993321191
Precision: 0.4585635359116022
Recall: 0.5366379310344828
F1: 0.4945382323733863
ROC-AUC: 0.7948993761296441
```

Step 7.4: Final evaluation — test set (XGBoost frozen)

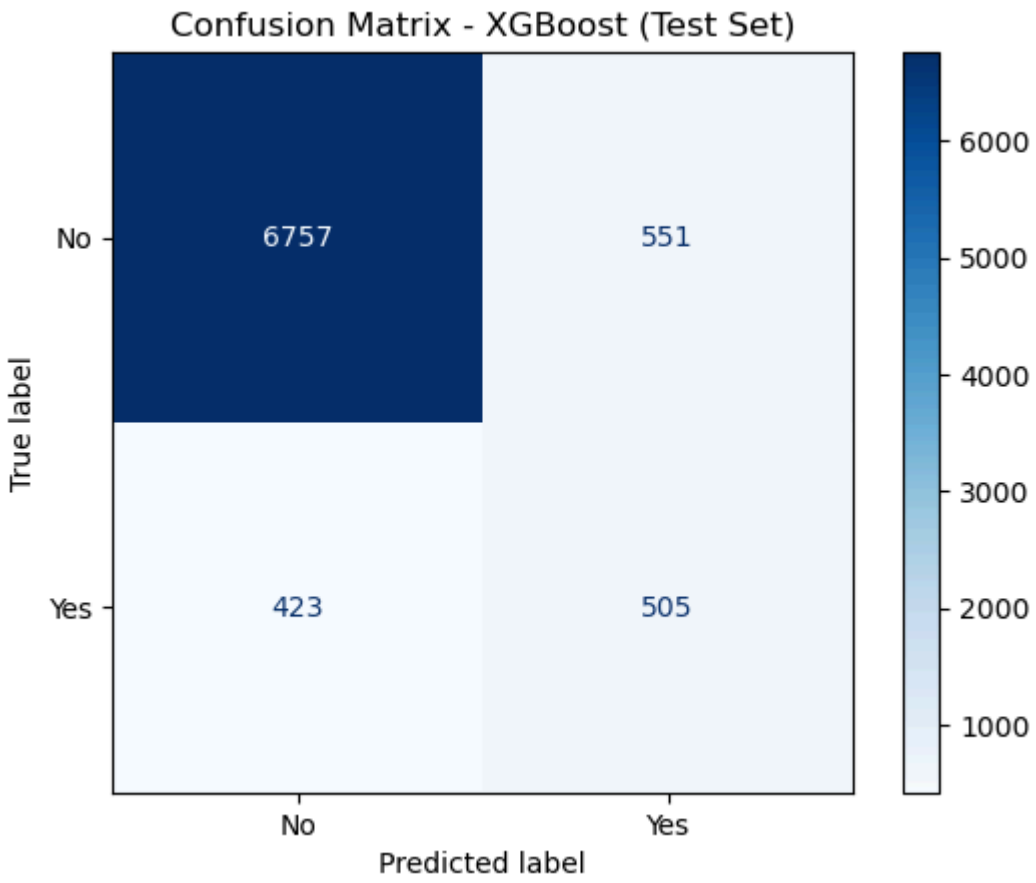
In [57]:
```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

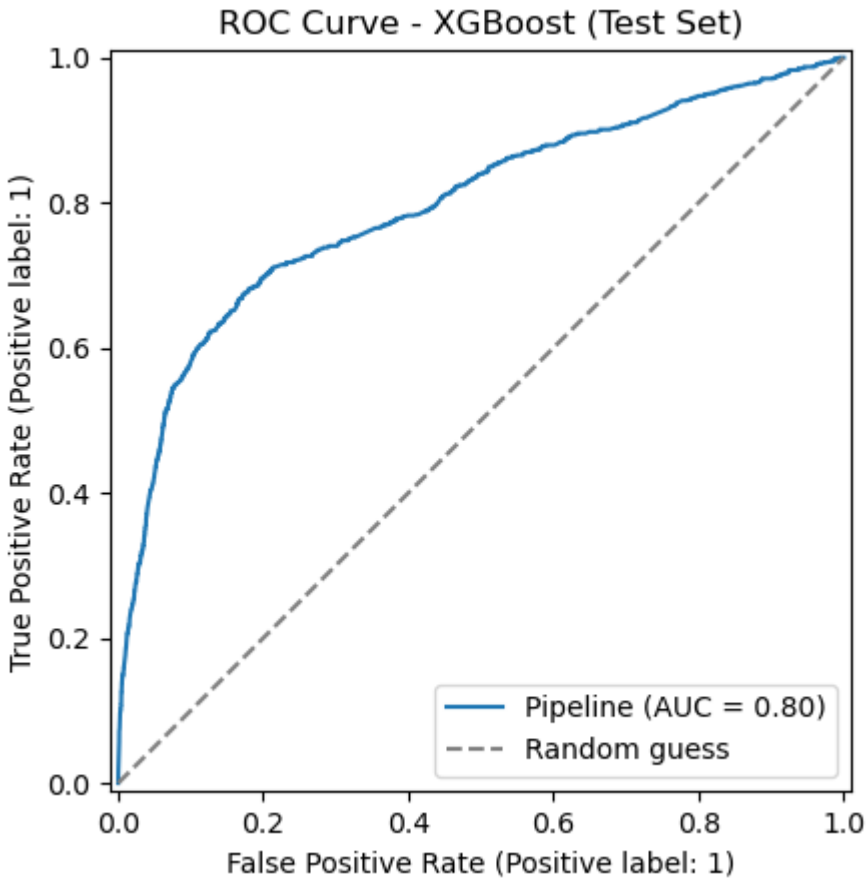
evaluate(xgb_final, X_test_tree, y_test, "Test set (final XGBoost evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(xgb_final, X_test_tree, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - XGBoost (Test Set)")
plt.show()

# ROC-AUC curve
RocCurveDisplay.from_estimator(xgb_final, X_test_tree, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - XGBoost (Test Set)")
plt.legend()
plt.show()
```

```
Test set (final XGBoost evaluation)
Accuracy: 0.8817387081107334
Precision: 0.47821969696969696
Recall: 0.5441810344827587
F1: 0.5090725806451613
ROC-AUC: 0.7966666489723119
```


Confusion Matrix - XGBoost (Test Set)
6757
551
423
505
No
Yes
True label
No
Yes
Predicted label
6000
5000
4000
3000
2000
1000


ROC Curve - XGBoost (Test Set)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
Pipeline (AUC = 0.80)
Random guess

Step 7: Model Comparison

A model comparison between the three algorithms was conducted to identify the best-performing model, using F1-score and ROC-AUC as the primary metrics because of the imbalanced dataset.

Each model was compared on the validation and test sets, ranked by the average of its F1-score and ROC-AUC, and checked for overfitting by measuring the gap between validation and test performance. XGBoost achieved the best overall ranking and was selected as the final model. The three models performed very similarly on ROC-AUC, and none showed signs of overfitting.

Complete results are in Table D.10 of the dissertation report (`report/Dissertation_Report.pdf`).

In [62]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
import pandas as pd

def get_metrics(model, X, y):
    pred = model.predict(X)
    proba = model.predict_proba(X)[:, 1]
    return {
        'Accuracy': accuracy_score(y, pred),
        'Precision': precision_score(y, pred),
        'Recall': recall_score(y, pred),
        'F1': f1_score(y, pred),
        'ROC-AUC': roc_auc_score(y, proba)
    }

# Score all three models on one dataset (validation or test)
def score_models(X_lr, X_tree, y):
    return pd.DataFrame({
        'Logistic Regression': get_metrics(lr_final, X_lr, y),
        'Random Forest': get_metrics(rf_final, X_tree, y),
        'XGBoost': get_metrics(xgb_final, X_tree, y)
    }).T.round(4)

# Step 1: score on validation data (used to pick the model)
val_df = score_models(X_validation_lr, X_validation_tree, y_validation)

# Step 2: score on test data (final, unseen data)
```

```
test_df = score_models(X_test_lr, X_test_tree, y_test)

# Step 3: rank by the average of F1 and ROC-AUC (best two metrics for imbalanced data)
test_df['F1 + ROC-AUC (avg)'] = ((test_df['F1'] + test_df['ROC-AUC']) / 2).round(4)
test_df = test_df.sort_values('F1 + ROC-AUC (avg)', ascending=False)

# Step 4: check overfitting — how much each score drops from validation to test
gap_df = (val_df - test_df[val_df.columns]).round(4)

# Print tables
print("Table X.X — Validation Performance\n", val_df, "\n")
print("Table X.X — Test Performance (Final)\n", test_df, "\n")
print("Table X.X — Final Ranking (F1 + ROC-AUC average)\n", test_df[['F1', 'ROC-AUC', 'F1 + ROC-AUC (avg)']], "\n")
print("Table X.X — Validation–Test Gap\n", gap_df, "\n")
print("Final selected model:", test_df['F1 + ROC-AUC (avg)'].idxmax())
```

```
Table X.X — Validation Performance
                     Accuracy  Precision  Recall      F1  ROC-AUC
Logistic Regression    0.8283     0.3529  0.6282  0.4519   0.8003
Random Forest          0.8770     0.4608  0.5388  0.4968   0.7991
XGBoost                0.8764     0.4586  0.5366  0.4945   0.7949

Table X.X — Test Performance (Final)
                     Accuracy  Precision  Recall      F1  ROC-AUC  \
XGBoost                0.8817     0.4782  0.5442  0.5091   0.7967
Random Forest          0.8798     0.4708  0.5377  0.5020   0.7977
Logistic Regression    0.8299     0.3590  0.6487  0.4622   0.7965

                    F1 + ROC-AUC (avg)
XGBoost                         0.6529
Random Forest                   0.6498
Logistic Regression             0.6294

Table X.X — Final Ranking (F1 + ROC-AUC average)
                         F1  ROC-AUC  F1 + ROC-AUC (avg)
XGBoost              0.5091   0.7967             0.6529
Random Forest        0.5020   0.7977             0.6498
Logistic Regression  0.4622   0.7965             0.6294

Table X.X — Validation–Test Gap
                     Accuracy  Precision  Recall      F1  ROC-AUC
Logistic Regression   -0.0016    -0.0061 -0.0205 -0.0103   0.0038
Random Forest         -0.0028    -0.0100  0.0011 -0.0052   0.0014
XGBoost               -0.0053    -0.0196 -0.0076 -0.0146  -0.0018

Final selected model: XGBoost
```

Step 8: SHAP Analysis

SHAP (SHapley Additive exPlanations) was applied to the final XGBoost model on the test set to explain which features influenced its predictions. The mean absolute SHAP value ranks each feature by its average impact on the model's output.

Key Takeaways: The four most influential features were `nr.employed` (employment level), `contact` (contact method), `emp.var.rate` (employment variation rate) and `poutcome` (outcome of the previous campaign). Demographic features such as age and job had much less

influence.

Business Insight: The model's predictions depend mainly on the economic climate, the contact channel and past campaign history. This suggests prioritising customers with a successful previous outcome, using the more effective contact channel and taking economic conditions into account when timing campaigns.

Complete results are in Table D.12 and the SHAP figures of the dissertation report (`report/Dissertation_Report.pdf`).

In [64]:
```
import shap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Pull the raw XGBoost model out of the pipeline
xgb_model = xgb_final.named_steps['classifier']

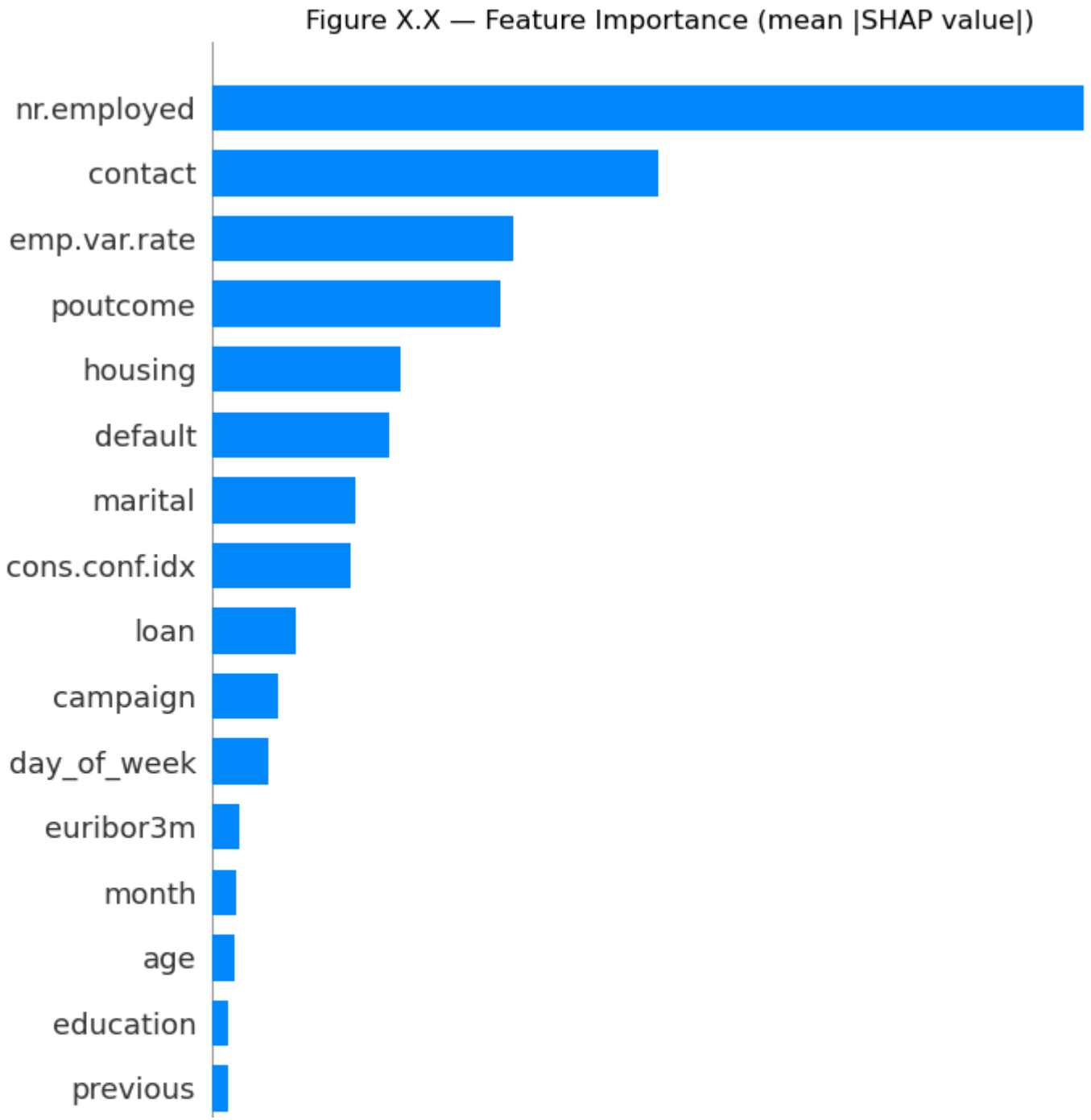
# 2. TreeExplainer on the bare model
explainer = shap.TreeExplainer(xgb_model)

# 3. SHAP values on the test set
#    SMOTE only affects training, so it does NOT transform test data —
#    the test features go straight to the classifier unchanged.
shap_values = explainer.shap_values(X_test_tree)

# 4. Bar plot — overall importance
shap.summary_plot(shap_values, X_test_tree, plot_type="bar", show=False)
plt.title("Figure X.X — Feature Importance (mean |SHAP value|)")
plt.tight_layout()
plt.show()

# 5. Ranked table
importance = pd.DataFrame({
    'Feature': X_test_tree.columns,
    'Mean |SHAP|': np.abs(shap_values).mean(axis=0)
}).sort_values('Mean |SHAP|', ascending=False).reset_index(drop=True)
importance['Rank'] = importance.index + 1

print("Table X.X — SHAP Feature Importance Ranking\n")
print(importance.to_string(index=False))
```


Figure X.X — Feature Importance (mean |SHAP value|)
nr.employed
contact
emp.var.rate
poutcome
housing
default
marital
cons.conf.idx
loan
campaign
day_of_week
euribor3m
month
age
education
previous

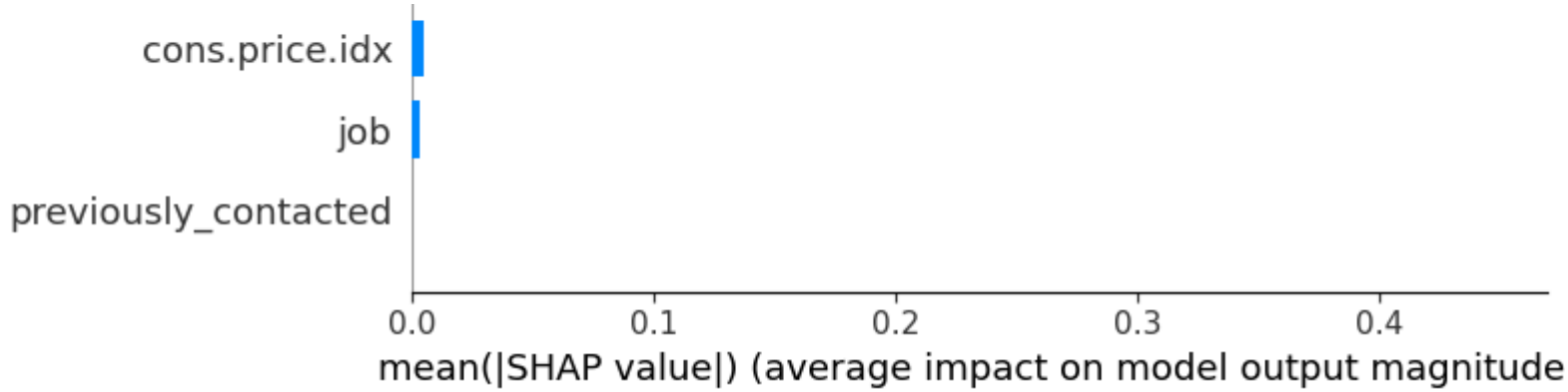


Table X.X — SHAP Feature Importance Ranking

Feature	Mean |SHAP|	Rank
nr.employed	0.447066	1
contact	0.228964	2
emp.var.rate	0.154315	3
poutcome	0.147819	4
housing	0.096710	5
default	0.090980	6
marital	0.073959	7
cons.conf.idx	0.070681	8
loan	0.043288	9
campaign	0.033532	10
day_of_week	0.028919	11
euribor3m	0.013738	12
month	0.012434	13
age	0.011914	14
education	0.008215	15
previous	0.008199	16
cons.price.idx	0.005150	17
job	0.003601	18
previously_contacted	0.000140	19